



Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

Date / /

SQL (Structured Query Language)

SQL ek language hai jiske use Database ke sath kaam krne ke liye use hota hai isse hum:

- Data ko store kr skte hai.
- Data ko fetch (nikal) skte hai.
- Data ko update k delete kr skte hai.

Data Base → Collection of Tables
Table = Column + Rows

Query → A Command to Database

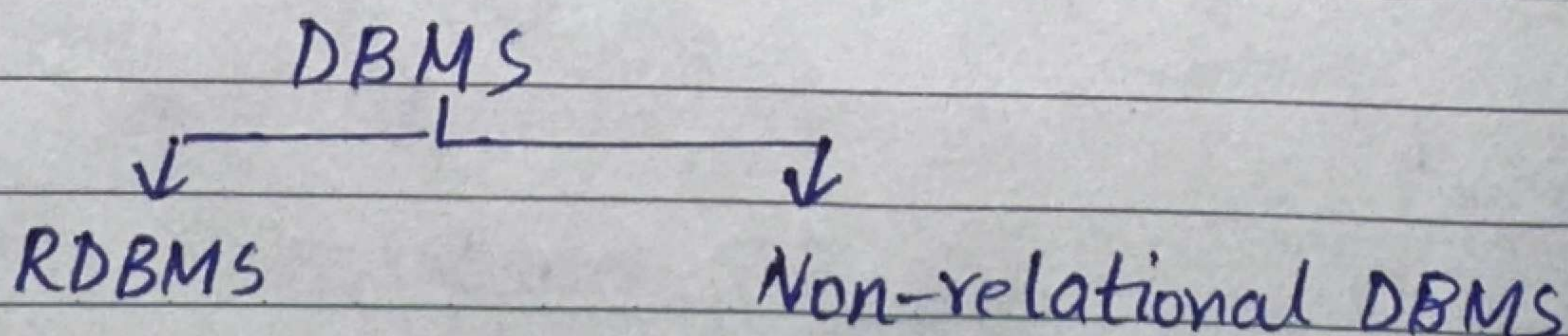
C Create

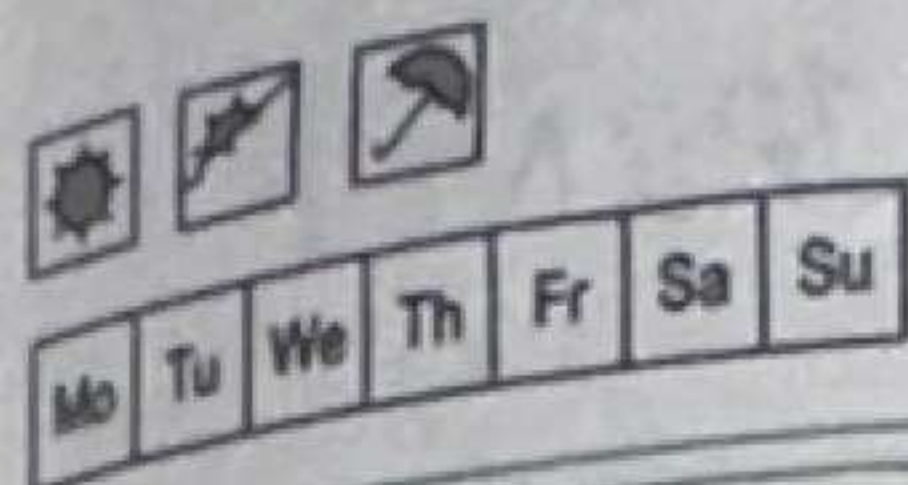
R Read

U Update

D Delete

Query ko Database mei dete hai phir database further database mai apply krke kaam krta hai





Date / /

R-DBMS (Relational Database Management System)

- follows a fixed & predefined data.
 - Data is organized into structured tables with rows & columns.
 - ensures data-integrity through constraint and relationships.
 - utilizes SQL for defining & manipulating data.
 - Well suited for ^{applications} ~~queries~~ requiring complex queries and transactional consistencies.
- Ex ÷ MySQL, Postgre SQL.



Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

ABHIRAL JAIN 07

Date / /

Non-relational Database Management system

- Provide flexible ~~and~~ data storage & modelling
- Doesn't enforce a fixed or consistent schema
- SQL is not imp. for data manipulation & querying
- use cases where
• Ideal for read operations are more frequent than write operations.

ex: MongoDB, Cassandra, neo4j.

DATATYPES EXPLAINED:

- **INT** Integer type, used for whole numbers
- **VARCHAR (100)** Variable length string, up to 100 characters
- **ENUM** A string object with a value chosen from a list of permitted value
ex: gender ENUM ("Male", "Female", "Other")

◦ **DATE** Stores ~~the~~ a date value
ex ÷ Date-of-birth DATE

◦ **TIMESTAMP** stores date & time, automatically set to current timestamp when row is created.

◦ **BOOLEAN** stores TRUE or FALSE values, often used for flags like is-active.

CONSTRAINTS EXPLAINED :

◦ **AUTO-INCREMENT** automatically generates a unique no. for each row

◦ **PRIMARY KEY** Uniquely identifies each row in ~~a column~~ the table

◦ **NOT NULL** Ensures a column cannot have NULL value

◦ **UNIQUE** Ensures all values in column are diff

◦ **DEFAULT** Sets a default value if no value provided



Mo Tu We Th Fr Sa Su

ABHIRAJAIN07

Date

/ /

Start SQL

Create Database

```
CREATE DATABASE startersql ;
```

↓
name of database

```
USE startersql ;
```

Create a Table

```
CREATE TABLE users (
```

```
  id int AUTO-INCREMENT PRIMARY KEY,
```

```
  name VARCHAR(100) NOT NULL,
```

```
  email VARCHAR(100) UNIQUE NOT NULL,
```

```
  gender ENUM ("M", "F", "O"),
```

```
  date-of-birth DATE,
```

```
  created-at TIMESTAMP DEFAULT CURRENT-  
TIMESTAMP;
```

```
);
```

```
SELECT * FROM users ;
```

DELETE THE DATABASE:

```
DROP DATABASE startersql ;
```



Mo Tu We Th Fr Sa Su

ABHIRAL JAIN 07

Date / /

RENAME A TABLE :

RENAME TABLE USERS TO PROGRAMMERS

ALTERING A TABLE :

~~alter~~ ALTER TABLE to modify existing table

add a column

ALTER TABLE USERS ADD COLUMN is-active

BOOLEAN DEFAULT TRUE;

DROP a Column

ALTER TABLE USERS DROP COLUMN is-active;

Modify Column

ALTER TABLE USERS MODIFY column email
VARCHAR(100) AFTER id;

Inserting data into MySQL Table

To add data, we use INSERT INTO statement



Insert without specifying column names

```
INSERT INTO US users VALUES  
(1, 'Alice', 'alice@.com', 'F', '1995-5-14',  
DEFAULT);
```

insert with specifying columns

```
INSERT INTO users (name, email, gender,  
date-of-birth) values  
( 'Bob', 'bob@gmail.com', 'M', 2003-02-24),  
( 'Ali', 'ali@gmail.com', 'F', 2005-03-30),  
( 'Abhi', 'abhi@gmail.com', 'M', 2005-09-15);
```

```
Select * from users ;
```

- If we can delete our database by mistake so we can get it by:

```
CREATE DATABASE IF NOT EXISTS startsql;
```

Querying Data

Querying data mai apn apni ~~defen~~ table ko dekh skte hai conditions apply kr ke.

The select statements is used to query data from table

SELECT column 1, column 2 from table-name;
users

To select all colbmnne

SELECT * FROM users;

FILTERING ROWS WITH WHERE

* Equal to

SELECT * FROM users WHERE gender = 'MALE';

* NOT Equal to

SELECT * FROM users WHERE gender $\neq$ 'MALE';

* GREATER THAN / LESS THAN

SELECT * FROM users WHERE date of birth < 1999-12-12;

SELECT * FROM users WHERE id > 10;

WORKING WITH NULL

* IS NULL

SELECT * FROM users WHERE date-of-birth IS NULL

* IS NOT NULL

SELECT * FROM users WHERE date-of-birth IS NOT NULL;

* BETWEEN

SELECT * FROM users WHERE date-of-birth between '1999-12-31' AND '2006-12-31'

* IN

SELECT * FROM users WHERE gender IN ('Male', 'Other');

* LIKE (Pattern Matching)

SELECT * FROM users WHERE name LIKE 'A%';

- Starts with A

SELECT * FROM users WHERE name LIKE '%a';

- ends with a

SELECT * FROM users WHERE name LIKE '%li%';

- contains 'li'

AND/OR

SELECT * FROM users WHERE gender = 'female'
AND date_of_birth > 2000-12-12;

SELECT * FROM users WHERE gender = 'MALE'
OR gender = 'Other';

ORDER BY

SELECT * FROM users ORDER BY date_of_birth ASC;

Ascending
Order

SELECT * FROM users ORDER BY name DESC;

Descending Order

LIMIT

select * FROM users LIMIT 5;
-- Top 5 rows

Select * FROM users ORDER BY created_at
DESC LIMIT 10;



UPDATE TABLE

~~WRONG SYNTAX~~

UPDATE users SET name = 'mahi', WHERE
id = 1;

// isse 1st wale ka naam change
ho jayega.

DELETION

DELETE FROM users WHERE id = 1;

• saare operators use kr skte hai

DELETE FROM users;

• ye krne se sab kuch delete ho jayega

FUNCTIONS -

helps to analyze, transform or summarize
data in tables.

SOL AGGREGATE FUNCTIONS

1) COUNT() count total users

```
SELECT COUNT (*) FROM users;
```

```
SELECT COUNT (*) FROM users WHERE gender='Male';
```

2) MIN() & MAX()

```
SELECT MIN(salary) AS min_salary, MAX(salary)
AS max_salary FROM users;
```

3) SUM()

```
SELECT SUM(salary) AS total_pay FROM users;
```

4) AVG()

```
SELECT AVG(salary) AS avg_salary FROM users;
```

GROUP BY

```
SELECT gender, AVG(salary) AS avg_salary
FROM users
```

```
GROUP BY gender;
```

STRING FUNCTION

ABHIRAJA LJA INO7

Date / /

1. ~~LEN~~ LENGTH()

length of user names:

```
SELECT name, LENGTH(name) AS name-length  
FROM users;
```

2. LOWER() & UPPER()

convert names to lower case & upper case

```
SELECT name, LOWER(name) AS lowercase_name  
FROM users;
```

```
SELECT name, UPPER(name) AS uppercase_name to FROM users;
```

3. ~~CON~~ CONCAT()

```
SELECT CONCAT(name, '<', email, '>') AS  
user-contact FROM users;
```

DATE FUNCTIONS

1. **NOW()** : current date & time.
`SELECT NOW();`

2. **YEAR()**, **MONTH()**, **DAY()**
extract parts of date-of-birth.
`SELECT name, YEAR(date-of-birth) AS birthyears
FROM users;`

3. **DATEDIFF()** find no. of days b/w birthdate
& today.

`SELECT name, DATEDIFF(CURDATE(), date-
-of-birth) AS At days
FROM users;`

TIMESTAMPDIFF() Calculate age in years

`SELECT name, TIMESTAMPDIFF(YEAR, date-of-birth,
CURDATE()) AS age FROM users;`



Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

ABHIRAJAN07

Date

/

/

MATHEMATICAL FUNCTIONS

1. ROUND(), FLOOR(), CEIL()

```
select SELECT SALARY,  
ROUND (salary) AS rounded,  
FLOOR (salary) AS floored,  
CEIL (salary) AS ceiled,  
FROM users;
```

2. MOD() find even or odd user ID's

```
SELECT id, MOD (id, 2) AS remainder FROM  
users;
```

CONDITIONAL FUNCTIONS

1) IF()

```
SELECT name, gender,  
IF (gender = 'Female', 'Yes', 'No') AS is-female  
FROM users;
```